JPA (Java Persistence API)

- là một chuẩn (specification) trong Java dùng để quản lý dữ liệu giữa ứng dụng Java và cơ sở dữ liệu quan hệ (RDBMS). Nó không phải là một framework riêng lẻ, mà là một tập hợp các quy tắc và API để các nhà cung cấp (như Hibernate, EclipseLink, OpenJPA) triển khai.
- **Chuẩn hóa**: JPA là chuẩn chính thức của Java EE (nay là Jakarta EE) cho persistence.
- **ORM (Object-Relational Mapping)**: Cho phép ánh xạ (map) các class Java thành bảng trong cơ sở dữ liệu, và các thuộc tính thành cột.
- **Entity**: Mỗi class được đánh dấu bằng @Entity sẽ tương ứng với một bảng trong DB.
- **JPA** chỉ là chuẩn, còn việc thực thi do các framework như Hibernate

Hibernate

- **là một framework ORM (Object-Relational Mapping) nổi tiếng trong Java, giúp lập trình viên ánh xạ các đối tượng Java (class, thuộc tính) sang bảng và cột trong cơ sở dữ liệu quan hệ.**

HQL vs JPQL

- **HQL (Hibernate Query Language):**
- Ra đời trước JPA. Ngôn ngữ truy vấn hướng đối tượng của Hibernate, thao tác trên entity/thuộc tính thay vì bảng/cột. Có một số cú pháp đặc thù (insert, update, delete trực tiếp) mà JPQL chưa chuẩn hóa.

- **JPQL (Java Persistence Query Language):**
- Định nghĩa trong chuẩn JPA (Java EE 5, 2006). Cú pháp gần giống HQL nhưng portable — chạy được trên nhiều ORM (Hibernate, EclipseLink, OpenJPA…).

- **Quan hệ:** Hibernate coi JPQL và HQL là một — cùng dùng chung engine phân tích. Khi viết JPQL trong Hibernate, Hibernate vẫn xử lý bằng engine HQL. Thực tế: JPQL = HQL chuẩn hóa.

EntityManager vs Session

- **EntityManager (JPA chuẩn):** API trung tâm của JPA. Dùng persist(), merge(), remove(), find(). Tạo query: createQuery(), createNamedQuery(), createNativeQuery(). Portable, chạy trên nhiều ORM.

- **Session (Hibernate API):** API trung tâm của Hibernate. Tương tự EntityManager nhưng có thêm tính năng nâng cao: batch processing, scrollable results, stateless session, createCriteria().

- **Cách lấy Session từ EntityManager:**
- Session session = em.unwrap(Session.class);

1. EntityManager (JPA chuẩn)

•Là **API chuẩn của JPA** (Java Persistence API).

•Dùng để làm việc với database theo cách **portable** (đổi ORM khác vẫn chạy).

•Các hàm chính:

- `persist()` → thêm mới
- `merge()` → update
- `remove()` → xóa
- `find()` → tìm theo id
- `createQuery()` → JPQL
- `createNativeQuery()` → SQL thuần

👉 **Ý nghĩa**:

Bạn code theo chuẩn → không bị phụ thuộc vào Hibernate.

2. Session (Hibernate API)

•Là **API riêng của Hibernate** (không phải chuẩn JPA).

•Có tất cả chức năng giống `EntityManager`, **+ thêm nhiều thứ nâng cao**:

- Batch processing (xử lý hàng loạt, tối ưu performance)
- Scrollable results (duyệt dữ liệu lớn)
- StatelessSession (bỏ cache → nhanh hơn)
- `createCriteria()` (query kiểu object)

👉 **Ý nghĩa**:

Dùng khi cần **tối ưu performance hoặc tính năng đặc biệt của Hibernate**.

Các giao diện truy vấn của JPA

- Cách 1: Sử dụng JPQL (Phổ biến nhất)

- Query query = em.createQuery("SELECT b FROM Book b");
- List<Book> books = query.getResultList();

- Cú pháp rất giống SQL, nhưng thay vì dùng tên bảng/cột, bạn dùng tên lớp (Entity) và thuộc tính.

- Query chạy trên mô hình đối tượng chứ không trực tiếp trên bảng.

2. Phân tích đoạn code

```
Query query = em.createQuery("SELECT b FROM Book b");
List<Book> books = query.getResultList();
```

Ý nghĩa:

- `"SELECT b FROM Book b"`
  - `Book` → là **class Entity**, không phải table
  - `b` → biến đại diện cho object
- `getResultList()` → trả về danh sách object `Book`

SQL (truy vấn bảng)

```
SELECT * FROM books;
```

JPQL (truy vấn object)

```
SELECT b FROM Book b
```

👉 Khác nhau:

- SQL → làm việc với **table + column**
- JPQL → làm việc với **class + field**

Các giao diện truy vấn của JPA

- **Cach 2: Criteria API (Truy van dong)**
- Type-safe, khong dung chuoi JPQL. Phu hop khi tao query dong (nhieu dieu kien tuy chon). Compiler kiem tra loi thay vi luc runtime.

- CriteriaBuilder cb = em.getCriteriaBuilder();
- CriteriaQuery<Book> cq = cb.createQuery(Book.class);
- Root<Book> book = cq.from(Book.class);
- cq.select(book).where(cb.equal(book.get("author"), "Nguyen Nhat Anh"));
- TypedQuery<Book> q = em.createQuery(cq);
- List<Book> books = q.getResultList();

Mình sẽ diễn giải từng bước của đoạn code để bạn thấy rõ luồng xử lý:

java
```
CriteriaBuilder cb = em.getCriteriaBuilder();
```
Lấy đối tượng CriteriaBuilder từ EntityManager. Đây là công cụ để bắt đầu xây dựng truy vấn động.

java
```
CriteriaQuery<Book> cq = cb.createQuery(Book.class);
```
Tạo một truy vấn Criteria với kiểu kết quả là Book. Nghĩa là khi chạy, truy vấn sẽ trả về danh sách các entity Book.

java
```
Root<Book> book = cq.from(Book.class);
```
Xác định entity gốc của truy vấn. Đây chính là “match” giữa entity Book trong code và bảng Book trong cơ sở dữ liệu.

book đóng vai trò như alias trong JPQL (FROM Book b thì b chính là alias).

java
```
cq.select(book).where(cb.equal(book.get("author"), "Nguyen Nhat Anh"));
```
Chọn tất cả các bản ghi Book và thêm điều kiện lọc: author = "Nguyen Nhat Anh".

book.get("author") tham chiếu đến thuộc tính author của entity Book.

java
```
TypedQuery<Book> q = em.createQuery(cq);
```
Biên dịch truy vấn Criteria thành một TypedQuery để có thể thực thi.

java
```
List<Book> books = q.getResultList();
```
Thực thi truy vấn và trả về danh sách kết quả, chính là List<Book>.

Các giao diện truy vấn của JPA

- 📑 **Cách 3: Native SQL (SQL thuần)**
- **Khái niệm**
- **Native SQL** = truy vấn SQL trực tiếp trên bảng/cột trong database.
- JPA/Hibernate cho phép bạn chạy SQL thuần và ánh xạ kết quả về entity hoặc đối tượng tùy ý.

- Native SQL với Book
- EntityManager em = ...;
- Session session = em.unwrap(Session.class);

- // Native SQL query
- NativeQuery<Book> query = session.createNativeQuery(
- "SELECT * FROM BOOK WHERE AUTHOR = :author", Book.class
-);
- query.setParameter("author", "Nguyen Nhat Anh");

- List<Book> books = query.getResultList();